

Grupo Entrada, realizado por:

- Cristian David Chalaca Salas
- Juan Camilo Holguin Zuluaga

Código para Entrada:

<https://edaplayground.com/x/bZxf>

Caso de prueba **IN-001 — UART_StartBit_Detection**

Objetivo: Verificar que el módulo `uart_range_checker` detecta correctamente el start bit (transición $1 \rightarrow 0$ en `uart_rx`) e inicia la recepción activando `rx_busy`.

Procedimiento de Prueba:

1. Inicializar la simulación con `uart_rx = 1` (línea idle).
2. Activar `rstn = 1` para iniciar el sistema.
3. Enviar un byte UART válido usando la tarea del testbench:
 - Colocar `uart_rx = 0` durante un período de bit para simular el start bit.
4. Observar las señales internas del DUT (`rx_busy`, `bit_idx`, `clk_cnt`).
5. Medir el ciclo exacto en que `rx_d2` detecta la transición $1 \rightarrow 0$.
6. Verificar que:
 - `rx_busy` pasa de $0 \rightarrow 1$ inmediatamente después de detectar el start bit.
 - `clk_cnt` se carga con `CLKS_PER_BIT/2`.
 - `bit_idx` se reinicia a 0.

Resultado esperado:

- La transición `uart_rx: 1  $\rightarrow$  0` provoca que el módulo entre en estado de recepción.
- `rx_busy = 1` dentro del mismo ciclo en el que se detecta el start bit.
- El receptor comienza el muestreo del primer bit de datos en `CLKS_PER_BIT/2`.

Prioridad: 5

Especificación: El sistema debe detectar el start bit como una transición de alto a bajo, activar `rx_busy`, y preparar el muestreo a mitad de bit.

Caso de prueba **IN-002 — UART_DataShift_Correct**

Objetivo: Verificar que el módulo `uart_range_checker` recibe y almacena los bits del byte UART en el orden correcto (LSB-first) dentro del registro `rx_shift`.

Procedimiento de Prueba:

1. Inicializar `uart_rx = 1` y activar `rstn = 1`.
2. Enviar un byte conocido mediante la tarea `send_uart_byte(data)` del testbench.
 - Preferiblemente un patrón fácil de identificar, por ejemplo:
 - `8'b1010_1100`
 - `8'd172`
3. Observar la señal interna `rx_shift` en cada ciclo donde `bit_idx` incrementa.
4. Verificar que:
 - `rx_shift[0]` recibe el primer bit enviado.
 - `rx_shift[7]` recibe el último bit enviado.
 - Los valores coinciden exactamente con el patrón enviado.
5. Confirmar que, al final de los 8 bits, `rx_data == rx_shift`.

Resultado esperado:

- El receptor UART debe almacenar los bits del byte en orden LSB-first.
- `rx_shift` debe contener el mismo valor que la secuencia de bits enviada.
- `rx_data` debe tomar el mismo valor cuando `rx_done = 1`.

Prioridad: 5

Especificación: El receptor UART debe capturar los 8 bits de datos en el orden LSB-first, asegurando que el registro `rx_shift` y `rx_data` representen correctamente el byte recibido.

Caso de prueba **IN-003 UART_StopBit_Validation**

Objetivo: Verificar que el módulo `uart_range_checker` reconoce correctamente el stop bit (nivel lógico 1) al finalizar la recepción de un byte UART y genera el pulso `rx_done`.

Procedimiento de Prueba:

1. Iniciar la simulación con `uart_rx = 1` (línea UART en estado idle).
2. Activar `rstn = 1`.

3. Enviar un byte válido mediante `send_uart_byte(data)` del testbench.
 - La tarea ya incluye un stop bit correcto (`uart_rx = 1`).
4. Observar las señales internas:
 - `bit_idx`
 - `rx_busy`
 - `rx_done`
5. Confirmar que cuando `bit_idx == 8`, el módulo espera el stop bit.
6. Verificar que cuando `bit_idx == 9`:
 - `rx_busy` regresa a 0.
 - `rx_done` se activa por 1 ciclo de reloj.
 - `rx_data` toma el valor final del byte recibido.

Resultado esperado:

- El stop bit debe ser detectado como lógico alto (1).
- El módulo debe permitir la finalización correcta de la recepción.
- `rx_done` debe valer 1 exactamente un ciclo al finalizar `bit_idx == 9`.

Prioridad: 4

Especificación: El stop bit debe ser alto; si se detecta correctamente, la lógica debe completar la recepción y generar `rx_done = 1`.

Caso de prueba **IN-004 — UART_StopBit_Error**

Objetivo: Verificar que el módulo `uart_range_checker` detecta correctamente un stop bit inválido (nivel lógico 0) y NO finaliza la recepción, es decir, no activa `rx_done` y descarta el byte.

Procedimiento de Prueba:

1. Inicializar la simulación con `uart_rx = 1` (idle).
2. Activar `rstn = 1`.
3. Enviar un byte UART forzando el stop bit en 0.

Procedimiento manual del stop bit inválido:

```
uart_rx <= 0; // START
```

```

#(BIT_PERIOD);

// enviar 8 bits válidos
for (i = 0; i < 8; i++) begin
    uart_rx <= data[i];
    #(BIT_PERIOD);
end

uart_rx <= 0; // STOP BIT INVALIDO
#(BIT_PERIOD);

```

4. Observar señales internas:

- rx_busy
- bit_idx
- rx_done

5. Verificar que cuando bit_idx == 9 (momento del stop bit):

- rx_done permanece en 0
- rx_busy vuelve a 0 sin generar recepción válida
- rx_data no se actualiza con los valores del byte

6. Verificar que no se genera la transición típica de finalización de byte.

Resultado esperado:

- El módulo debe identificar que el stop bit es inválido (0).
- rx_done no debe activarse.
- La recepción debe terminar sin propagar el dato a rx_data.
- Debe descartarse el byte completo.

Prioridad: 3

Especificación: El stop bit debe ser alto; si se detecta bajo, debe considerarse como byte inválido y rx_done no debe generarse.

Caso de prueba **IN-005 — RX_Done_Assertion**

Objetivo: Verificar que el módulo uart_range_checker activa correctamente la señal rx_done exactamente por un ciclo de reloj una vez se completa la recepción de un byte válido.

Procedimiento de Prueba:

1. Inicializar la simulación con `uart_rx = 1` y activar `rstn = 1`.
2. Enviar un byte UART válido usando la tarea `send_uart_byte(data)`.
3. Monitorear las señales internas:
 - `bit_idx`
 - `rx_busy`
 - `rx_done`
 - `rx_data`
4. Verificar que durante la recepción:
 - `bit_idx` progresa de 0 a 9 correctamente.
 - En `bit_idx == 9`, el stop bit sea válido.
5. Observar que:
 - `rx_done` se activa (`rx_done = 1`) solo durante un ciclo de reloj al finalizar `bit_idx == 9`.
 - `rx_done` vuelve inmediatamente a 0 en el siguiente ciclo.
6. Confirmar además que `rx_data` contiene el byte recibido, pero no se actualiza de nuevo después del pulso de `rx_done`.

Resultado esperado:

- `rx_done` debe activarse exactamente una vez por byte recibido.
- Debe durar un solo ciclo de reloj.
- Debe ocurrir únicamente cuando se completa la recepción válida del byte.
- `rx_data` debe coincidir con el byte enviado.

Prioridad: 5

Especificación: Al finalizar la recepción del stop bit válido, el módulo debe generar un pulso de `rx_done` de un ciclo indicando disponibilidad de datos.

Caso de prueba **IN-006 — RangeCheck_ValidValue**

Objetivo: Verificar que el módulo `uart_range_checker` reconoce correctamente los valores dentro del rango permitido (`MIN_VALUE` a `MAX_VALUE`, es decir, 0 a 127) y enciende `led_ok`, manteniendo `led_err` apagado.

Procedimiento de Prueba:

1. Activar rstn = 1 para iniciar el módulo.
2. Enviar varios bytes UART válidos con valores dentro del rango permitido:
 - 0
 - 1
 - 50
 - 65
 - 100
 - 127
3. Para cada valor recibido:
 - Verificar que rx_done = 1.
 - Observar que rx_data contiene el valor correcto.
 - Comprobar que:
 - led_ok = 1
 - led_err = 0
4. Repetir el procedimiento con diferentes valores del rango para verificar estabilidad.

Resultado esperado:

- Cada vez que se recibe un valor dentro de 0 a 127:
 - led_ok debe encenderse.
 - led_err debe permanecer apagado.
 - rx_data debe contener el valor recibido.
- El comparador debe funcionar correctamente sin falsos positivos.

Prioridad: 5

Especificación: Los valores dentro del rango permitido deben ser aceptados, generando led_ok = 1 y led_err = 0.

Caso de prueba **IN-007 — RangeCheck_InvalidValue**

Objetivo: Confirmar que el módulo `uart_range_checker` detecta correctamente los valores fuera del rango permitido (128–255) y activa `led_err`, manteniendo `led_ok` apagado.

Procedimiento de Prueba:

1. Activar `rstn = 1` para iniciar el sistema.
2. Enviar valores UART válidos desde el punto de vista del protocolo, pero inválidos por rango, tales como:
 - 128
 - 150
 - 200
 - 255
3. Para cada valor recibido:
 - Verificar que `rx_done = 1` (el byte fue recibido correctamente por el protocolo).
 - Comprobar que:
 - `led_err = 1`
 - `led_ok = 0`
 - Verificar que el comparador bloquea la validación del valor.
4. Repetir la prueba con diferentes valores fuera de rango para comprobar consistencia.

Resultado esperado:

- Para valores fuera de 128–255:
 - `led_err` se activa.
 - `led_ok` permanece en 0.
 - Aunque `rx_data` toma el valor recibido, no se considera válido.
- El sistema no debe producir falsos positivos en los LEDs.

Prioridad: 5

Especificación: Los valores fuera del rango permitido deben marcarse como inválidos, activando `led_err = 1` y desactivando `led_ok`.

Caso de prueba **IN-008 — SaveValue_OnValid**

Objetivo: Verificar que el módulo `top_uart_demux` almacena correctamente en `valor_reg` únicamente los valores válidos recibidos por UART (0 a 127) cuando `rx_done = 1` y `led_ok = 1`.

Procedimiento de Prueba:

1. Activar `rstn = 1` para iniciar el sistema.
2. Enviar un valor UART dentro del rango permitido, por ejemplo:
 - 65
 - 100
 - 20
3. Observar las señales relevantes:
 - `rx_done`
 - `rx_data`
 - `led_ok`
 - `valor_reg`
4. Confirmar que:
 - Cuando `rx_done = 1` y `led_ok = 1`:
→ `valor_reg` debe actualizarse con los 7 bits inferiores de `rx_data`.
5. Verificar la actualización:
 - Para un envío de 65 (0x41):
`valor_reg` debe tomar `65[6:0] = 65`.
 - Repetir con otros valores válidos.

Resultado esperado:

- Cada vez que el UART recibe un valor válido:
 - `led_ok = 1`
 - `rx_done = 1`

- valor_reg se actualiza correctamente con los 7 bits inferiores del valor recibido.
- El valor almacenado debe coincidir exactamente con el byte recibido (menos el bit 7 si este se usa para rango).

Prioridad: 4

Especificación: valor_reg debe actualizarse únicamente cuando rx_done es alto y el valor es válido, reflejando los bits [6:0] de rx_data.

Caso de prueba **IN-009 — BlockSave_OnInvalid**

Objetivo: Verificar que el módulo top_uart_demux NO actualiza el registro valor_reg cuando se recibe un valor UART fuera de rango (128–255), incluso si el protocolo UART es correcto.

Procedimiento de Prueba:

1. Inicializar el sistema activando rstn = 1.
2. Forzar un valor inicial en valor_reg enviando un valor válido (por ejemplo, 65).
3. Guardar mentalmente ese valor como referencia (o verlo en la forma de onda).
4. Enviar un valor inválido por UART, por ejemplo:
 - 128
 - 200
 - 255
5. Observar las señales relevantes:
 - rx_done
 - rx_data
 - led_err
 - valor_reg
6. Verificar que:
 - rx_done = 1 (recepción UART correcta)
 - led_err = 1

- valor_reg NO cambia respecto al valor previo válido

7. Repetir con otros valores fuera de rango para confirmar la robustez.

Resultado esperado:

- Al recibir un valor fuera de rango:
 - led_err debe encenderse.
 - led_ok debe permanecer en 0.
 - valor_reg debe mantener su valor previo.
- No debe ocurrir actualización accidental del registro.

Prioridad: 4

Especificación: El registro valor_reg debe actualizarse solo cuando rx_done = 1 y led_ok = 1. Cuando led_err = 1, no debe modificarse.

Caso de prueba IN-010 — Demux_Select_00

Objetivo: Verificar que cuando perilla = 2'b00, el módulo demux_4salidas dirige el valor valor_reg únicamente a la salida sal0, y que las demás salidas permanecen en cero.

Procedimiento de Prueba:

1. Asegurarse de que valor_reg contiene un valor válido (por ejemplo, 65).

Configurar:

perilla = 2'b00;

2. Observar las señales:

- sal0
- sal1
- sal2
- sal3

3. Verificar que:

- sal0 == valor_reg
- sal1 == 0
- sal2 == 0

- sal3 == 0
- 4. Repetir con diferentes valores de valor_reg para validar que el comportamiento es consistente.

Resultado esperado: Cuando perilla = 00, el módulo debe enrutar el valor de entrada exclusivamente hacia sal0, manteniendo las otras tres salidas apagadas.

Prioridad: 3

Especificación: El DEMUX debe activar únicamente la salida seleccionada por perilla; para 00, debe activarse únicamente sal0.

Caso de prueba **IN-011 — Demux_Select_01**

Objetivo: Verificar que cuando perilla = 2'b01, el módulo demux_4salidas dirige el valor valor_reg únicamente a la salida sal1, y que las demás salidas permanecen en cero.

Procedimiento de Prueba:

1. Asegurarse de que valor_reg contiene un valor válido (por ejemplo, 100).

Configurar:

perilla = 2'b01;

2. Monitorear las señales:

- sal0
- sal1
- sal2
- sal3

3. Verificar que:

- sal1 == valor_reg
- sal0 == 0
- sal2 == 0
- sal3 == 0

4. Repetir con distintos valores en valor_reg (p. ej. 5, 64, 120) para validar consistencia del comportamiento.

Resultado esperado: Cuando perilla = 01, el módulo debe enrutar el valor de entrada exclusivamente hacia sal1, manteniendo las otras salidas en cero.

Prioridad: 3

Especificación: El DEMUX debe enviar el valor a la salida seleccionada por perilla; para 01, debe activarse únicamente sal1.

Caso de prueba **IN-012 — Demux_Select_10**

Objetivo: Verificar que cuando perilla = 2'b10, el módulo demux_4salidas enruta correctamente el valor valor_reg únicamente hacia la salida sal2, manteniendo el resto de salidas en cero.

Procedimiento de Prueba:

1. Asegurarse de que valor_reg contiene un valor válido (por ejemplo, 50).

Configurar:

perilla = 2'b10;

2. Observar las señales del demux:

- sal0
- sal1
- sal2
- sal3

3. Confirmar que:

- sal2 == valor_reg
- sal0 == 0
- sal1 == 0
- sal3 == 0

4. Repetir la prueba usando diferentes valores en valor_reg (p.ej., 12, 80, 127) para comprobar estabilidad.

Resultado esperado: Cuando perilla = 10, el valor debe dirigirse únicamente a sal2, y ninguna de las otras salidas debe recibir contenido.

Prioridad: 3

Especificación: Para una selección 10, solo la salida sal2 debe recibir el valor; las demás se mantienen en cero según el comportamiento de un demultiplexor de 1 a 4.

Caso de prueba **IN-013 — Demux_Select_11**

Objetivo: Verificar que cuando `perilla = 2'b11`, el módulo `demux_4salidas` enruta el valor `valor_reg` únicamente a la salida `sal3`, y que las demás salidas permanecen en cero.

Procedimiento de Prueba:

1. Asegurarse de que `valor_reg` contiene un valor válido (por ejemplo, 77).

Ajustar:

`perilla = 2'b11;`

2. Monitorizar las señales:

- `sal0`
- `sal1`
- `sal2`
- `sal3`

3. Verificar que:

- `sal3 == valor_reg`
- `sal0 == 0`
- `sal1 == 0`
- `sal2 == 0`

4. Repetir con distintos valores en `valor_reg` (p. ej. 1, 63, 127) para comprobar consistencia del enrutamiento.

Resultado esperado: Con `perilla = 11`, sólo `sal3` debe recibir el valor; las otras salidas deben permanecer en cero.

Prioridad: 3

Especificación: El DEMUX debe activar únicamente la salida seleccionada por `perilla`; para 11, solo `sal3` debe activarse.

Caso de prueba **IN-014 — DEMUX_ZeroOnInvalid**

Objetivo: Verificar que cuando se recibe un valor fuera de rango (128–255), el registro `valor_reg` no se actualiza y, como consecuencia, todas las salidas del DEMUX permanecen en cero, independientemente del valor de `perilla`.

Procedimiento de Prueba:

1. Iniciar el sistema activando `rstn = 1`.

2. Cargar un valor válido inicial en `valor_reg`, por ejemplo 65.
3. Enviar un valor inválido por UART, por ejemplo:
 - 128
 - 150
 - 200
 - 255
4. Confirmar:
 - `rx_done = 1`
 - `led_err = 1`
 - `valor_reg` no cambia (debe seguir en 65 u otro valor previo válido)
5. Configurar diferentes valores en perilla:
 - 00, 01, 10, 11
6. Verificar que:
 - Todas las salidas (`sal0`, `sal1`, `sal2`, `sal3`) permanecen en 0, porque el DEMUX recibe un `valor_reg` que no cambió respecto al valor anterior, o que se encuentra en cero si era el estado inicial.
7. Repetir con múltiples valores inválidos.

Resultado esperado:

- Para valores fuera de rango:
 - `valor_reg` no se actualiza.
 - El DEMUX no propaga valores inválidos.
 - Todas las salidas permanecen en 0 o con el valor previamente latched (si antes había un valor válido).
- La salida final corresponde únicamente al último valor válido, y nunca a valores inválidos.

Prioridad: 4

Especificación: El DEMUX no debe propagar valores inválidos, ya que `valor_reg` solo se actualiza cuando `led_ok = 1`.

Caso de prueba **IN-015 — ResetBehavior**

Objetivo: Verificar que al activar el reset ($\text{rstn} = 0$), todos los registros y señales internas de los módulos (`uart_range_checker`, `top_uart_demux`, `demux_4salidas`) vuelvan a su estado inicial definido y que, al desactivar el reset ($\text{rstn} = 1$), el sistema arranque adecuadamente.

Procedimiento de Prueba:

1. Establecer $\text{rstn} = 0$ al iniciar la simulación.
2. Observar los siguientes registros y señales internas:
 - En `uart_range_checker`:
 - $\text{rx_busy} = 0$
 - $\text{clk_cnt} = 0$
 - $\text{bit_idx} = 0$
 - $\text{rx_done} = 0$
 - $\text{rx_data} = 0$
 - $\text{led_ok} = 0$
 - $\text{led_err} = 0$
 - En `top_uart_demux`:
 - $\text{valor_reg} = 0$
 - En el `demux_4salidas`:
 - $\text{sal0} = 0$
 - $\text{sal1} = 0$
 - $\text{sal2} = 0$
 - $\text{sal3} = 0$

Después de verificar los valores iniciales, activar el sistema:

```
 $\text{rstn} = 1;$ 
```

3. Enviar un byte válido por UART para comprobar que el sistema arranca correctamente y responde de forma normal.
4. Confirmar:
 - No quedan señales trabadas por el reset.
 - La lógica se comporta según especificación después del reset.

Resultado esperado:

- Durante $rstn = 0$, todas las señales críticas deben tomar sus valores iniciales conocidos.
- Después de $rstn = 1$, el sistema debe:
 - Iniciar la detección de start bit.
 - Recibir valores.
 - Validar rango.
 - Actualizar `valor_reg` solo si corresponde.
 - Enrutar correctamente por el DEMUX.

Prioridad: 4

Especificación: Todos los registros deben inicializarse correctamente durante el reset y el sistema debe operar normalmente al salir del reset.

Caso de prueba **IN-016 — UART_MidBitSampling**

Objetivo: Confirmar que el receptor UART muestrea cada bit en el punto medio del periodo, es decir, a $CLKS_PER_BIT/2$ después del start bit, tal como lo implementa el módulo `uart_range_checker`.

Este muestreo centrado es crucial para evitar errores ocasionados por jitter, baja precisión en el baud rate o señales con borde lento.

Procedimiento de Prueba:

1. Configurar la simulación con los valores reales:
 - $CLK_FREQ = 10\text{ MHz}$ o el que estés usando.
 - $BAUD_RATE = 9600$ o el que uses en la prueba.
2. Forzar un byte UART de manera controlada usando la tarea `send_uart_byte` del `testbench`.

3. Abrir la forma de onda y observar:

- El instante exacto en el que el receptor detecta el start bit (cuando `uart_rx` pasa de 1 → 0).

Verificar que, tras detectar el start bit, el módulo espera:

`CLKS_PER_BIT / 2` ciclos

- antes de tomar la primera muestra.

Confirmar que cada transición de muestreo para bits posteriores ocurre cada:

`CLKS_PER_BIT` ciclos

- después de la primera muestra.

4. Observar las señales:

- `rx_busy`
- `clk_cnt`
- `bit_idx`
- `rx_shift`

5. Confirmar que:

- El bit capturado en `rx_shift[n]` corresponde al valor correcto del bit enviado.
- No hay corrimiento temporal en la captura.

Resultado esperado:

- El módulo:
 1. Detecta el start bit correctamente.

Espera el tiempo definido:

`CLKS_PER_BIT/2`

2. (mitad del bit).
3. Muestrea los 8 bits correctamente en orden.
4. `rx_shift` contiene el valor exacto, sin errores de tiempo.
5. Al completar, `rx_done = 1` un solo ciclo.

Prioridad: 3

Especificación: El receptor UART debe muestrear cada bit en el centro del periodo de bit para garantizar exactitud bajo condiciones de jitter y ruido. El valor central usado es:

$\text{CLKS_PER_BIT} / 2$

Caso de prueba **IN-017 — UART_NoiseStability**

Objetivo: Verificar que el receptor UART sea robusto ante ruido o fluctuaciones esporádicas en `uart_rx`, asegurando que no se inicie una recepción falsa por pulsos breves de ruido (glitches).

Esto valida la efectividad de la doble sincronización implementada en:

```
rx_d1 <= uart_rx;
```

```
rx_d2 <= rx_d1;
```

Gracias a esto el sistema rechaza pulsos muy cortos.

Procedimiento de prueba:

1. Ejecutar una simulación normal con clock y reset.
2. Enviar pulsos de ruido en `uart_rx` sin que correspondan a un byte válido.

Ejemplos:

- Un pulso a 0 de duración menor a $\text{CLKS_PER_BIT}/4$.
- Alternancia rápida $1 \rightarrow 0 \rightarrow 1$ sobre `uart_rx`.
- Varias perturbaciones aleatorias antes de enviar un byte válido.

Ejemplo manual en el testbench:

```
uart_rx = 0; #500; // pulso demasiado corto para ser un start bit
```

```
uart_rx = 1; #2000;
```

3. Confirmar en la forma de onda que:
 - `rx_busy` no cambia a 1 durante los pulsos de ruido.
 - `bit_idx` permanece en cero.
 - `rx_shift` no se modifica.
 - `rx_done` nunca se activa.
4. Finalmente, enviar un byte UART válido usando `send_uart_byte()` y confirmar que:
 - El byte se recibe correctamente después del ruido.

Resultado esperado:

- Los pulsos de ruido no producen detección de start bit.

- rx_busy permanece en 0.
- rx_done permanece en 0.
- No se corrompen datos internos del receptor.
- La recepción real inicia únicamente cuando se detecta un start bit válido y estable.

Prioridad: 2

Especificación: El receptor UART debe mantener integridad frente a ruidos breves usando doble sincronización (rx_d1 y rx_d2), rechazando transiciones de alta frecuencia que no cumplen los tiempos mínimos de un start bit.